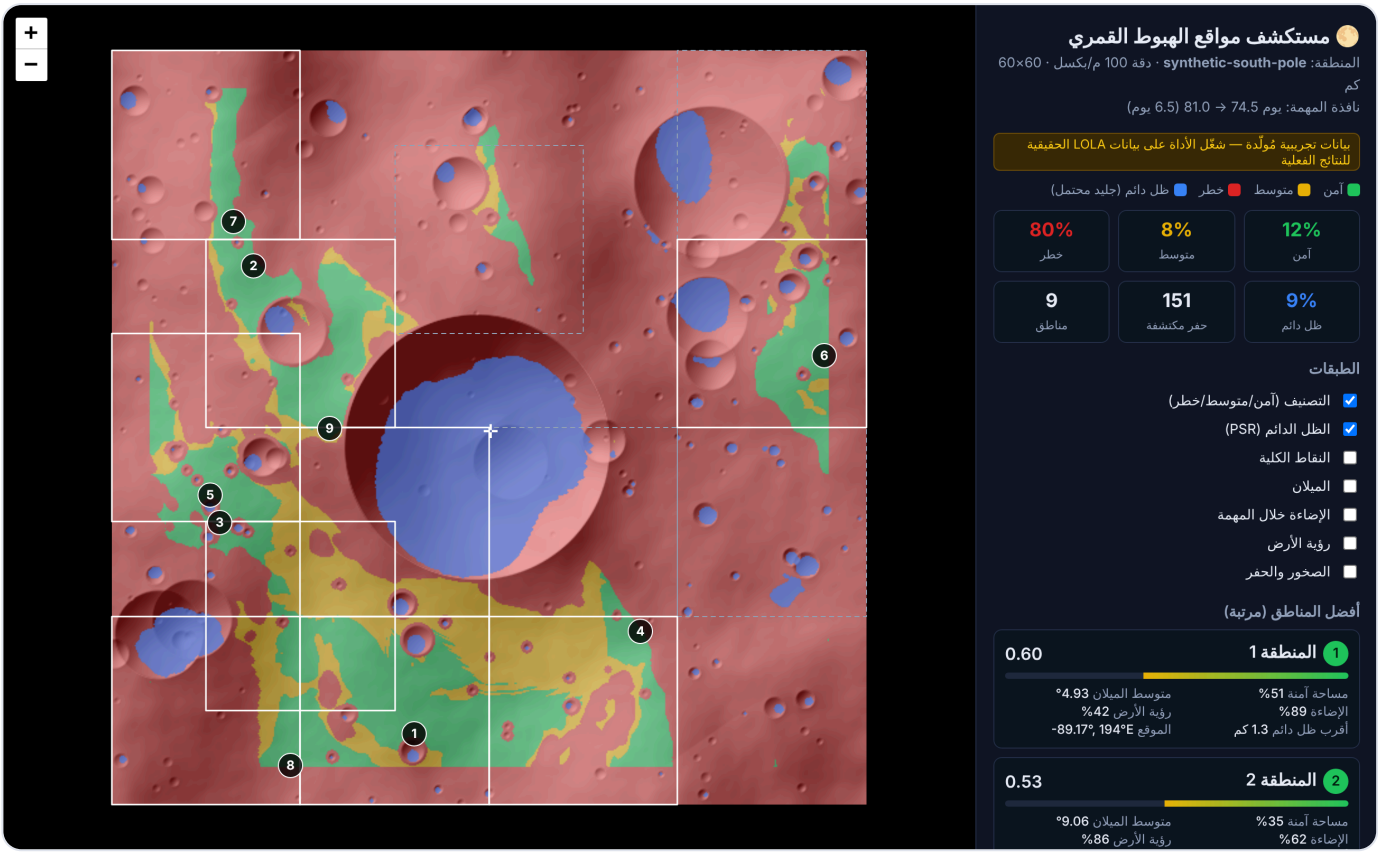
LUNAR LANDING SITE NAVIGATOR

○ مستكشف مواقع الهبوط القمري
الدليل الكامل من الألف إلى الياء

نظام يختار أفضل الأماكن لهبوط المركبات على القطب الجنوبي للقمر، بنفس الطريقة اللي استخدمتها ناسا في اختيار مناطق Artemis III



آمن | متوسط | خطر | ظل دائم (جليد محتمل)

المحتويات

1. فكرة المشروع
2. كيف ناسا تختار مكان الهبوط
3. شروط المكان الآمن
4. شكل النظام من فوق
5. محتويات المشروع (الملفات)
6. التثبيت على جهازك
7. أول تشغيل (البيانات التجريبية)
8. كيف تقرأ الخريطة والنتائج
9. التشغيل على سيرفرات GitHub
10. التشغيل على بيانات ناسا الحقيقية
11. كيف يشتغل النظام من الداخل
12. تعديل الشروط والأوزان
13. كشف الحفر والصخور بـ YOLO
14. الاختبارات
15. حدود النظام وأفكار للتطوير
16. حل المشاكل الشائعة
17. المصادر

1. فكرة المشروع

قبل ما تهبط أي مركبة على القمر لازم نعرف وين المكان الآمن. القطب الجنوبي بالذات صعب: الشمس دايمًا قريبة من الأفق، فالظلال طويلة جدًا، وفيه حفر عميقة ما تدخلها الشمس أبد. وهذي الحفر نفسها هي اللي يُحتمل فيها **جليد ماء**، يعني أهم مكان علميًا وأصعب مكان هندسيًا.

المشروع ياخذ خريطة الارتفاعات حقت القطب الجنوبي، ويحسب لكل نقطة فيها:

- كم ميلانها
- كم نسبة الوقت اللي تكون فيها تحت الشمس خلال المهمة
- كم نسبة الوقت اللي تكون فيها الأرض ظاهرة، عشان الاتصال
- كم تبعد عن أقرب حفرة مظلمة دايمًا
- كم فيها حفر وصخور

بعدها يعطي كل نقطة نقاط حسب أوزان معينة، ويقسم المنطقة لمربعات 15×15 كم، ويرتب أفضل 9 مناطق، ويعرضها على خريطة تفاعلية.

2. كيف ناسا تختار مكان الهبوط

عند ناسا مركبة اسمها (Lunar Reconnaissance Orbiter) **LRO** تدور حول القمر من 2009، وتجمع البيانات بثلاث أدوات رئيسية:

الأداة	وش تقيس	وش يقابلها في مشروعنا
LOLA ليزر	ترسل نبضات ليزر للسطح وتقيس الارتفاع بدقة عالية، ومنها تطلع خرائط الارتفاع والميلان والإضاءة	`terrain.py` + `illumination.py`
Diviner حرارة	تقيس حرارة السطح. الصخور تبرد أبطأ من التراب في الليل، فتنكشف المناطق الصخرية	الخيار `--rocks` في `hazards.py`
LROC كاميرات	كاميرا NAC دقتها حوالي 0.5 م للبكسل، تشوف الحفر الصغيرة والصخور واحدة واحدة	YOLO في `hazards.py`

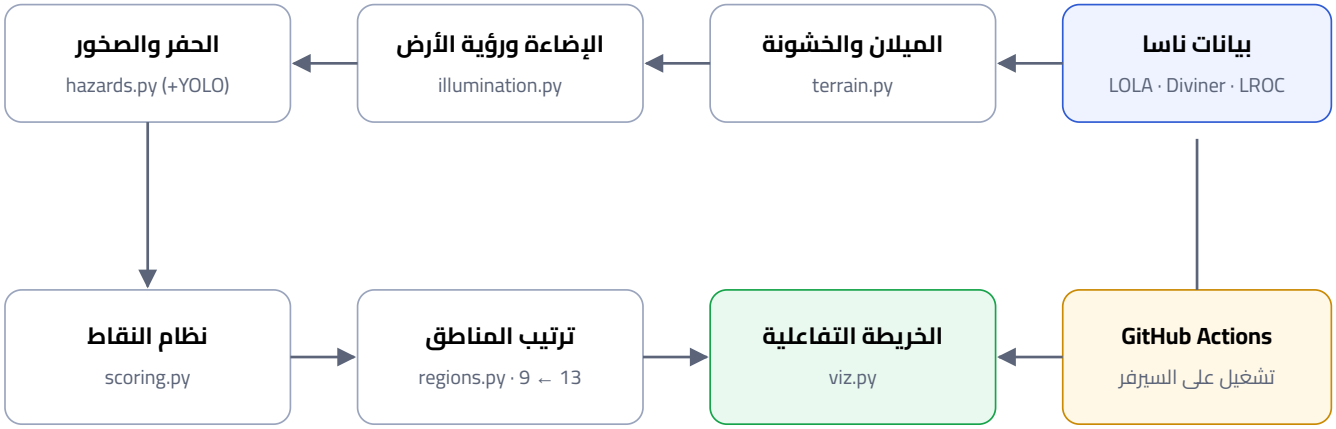
بهالبيانات حددت ناسا **13 منطقة** مرشحة قرب القطب الجنوبي لمهمة Artemis III، وبعدين قلّصتها لـ**9 مناطق**، وكل منطقة تقريبًا 15×15 كم. مشروعنا يمشي على نفس الخطوات: يختار 13 مرشحة، وبعدها يحتفظ بأفضل 9.

3. شروط المكان الآمن

الشرط	القيمة	ليش مهم
الميلان	أقل من 8° إلى 10°	إذا الأرض مايلة أكثر من كذا، المركبة ممكن تنقلب أو ما تثبت
الإضاءة	شمس أغلب مدة المهمة (حوالي 6.5 يوم)	الطاقة من الألواح الشمسية، والحرارة لازمة للأجهزة
الاتصال بالأرض	الأرض تكون فوق الأفق	الاتصال المباشر يحتاج خط نظر مفتوح للأرض
القرب من الظل الدائم	كم كيلومتر	عشان رواد الفضاء أو الروبوتات يوصلون للجليد
الصخور والحفر	أقل ما يمكن	صخرة وحدة تحت رجل المركبة ممكن تخرّب الهبوط

في التصنيف، النقطة تنحسب آمنة بس إذا كانت المنطقة كلها حولها آمنة في دائرة نصف قطرها 300 م، لأن المركبة ما تهبط في نقطة بالضبط، تهبط في منطقة اسمها "قطع الهبوط".

4. شكل النظام من فوق



مراحل النظام بالترتيب: من البيانات الخام إلى الخريطة النهائية

5. محتويات المشروع (الملفات)

```
lunar-landing-navigator/
├── lunar_nav/                ← الكود الأساسي
│   ├── config.py             ← الشروط والأوزان والثوابت الفيزيائية
│   ├── data.py               ← البيانات التجريبية + قراءة ملفات ناسا
│   ├── terrain.py            ← الميلان والخشونة والظل الفني للخريطة
│   ├── illumination.py       ← الأفق، الشمس، الأرض، الظل الدائم
│   ├── hazards.py            ← خريطة الخطر + YOLO + كشف الحفر
│   ├── scoring.py            ← النقاط والتصنيف (أخضر/أصفر/أحمر)
│   ├── regions.py            ← تقسيم المناطق وترتيبها وتحويل الإحداثيات
│   ├── pipeline.py           ← يربط كل المراحل ببعض
│   ├── viz.py                ← الخريطة التفاعلية وصورة الملخص
│   ├── cli.py                ← أوامر التشغيل
│   └── vendor/               ← مكتبة الخرائط Leaflet (عشان تشتغل بدون إنترنت)
├── scripts/download_nasa_data.py   ← تنزيل بيانات ناسا
├── tests/test_pipeline.py          ← الاختبارات
├── docs/guide/                     ← هذا الدليل
├── requirements.txt
└── README.md

.github/workflows/lunar-navigator.yml  ← التشغيل على سيرفرات GitHub
```

6. التثبيت على جهازك

المتطلبات

- **Python 3.10 أو أحدث**. تنزله من python.org، وفي ويندوز فعّل خيار "Add Python to PATH" وقت التثبيت.
- **Git**، عشان تنزل المشروع من GitHub. هذا اختياري، تقدر تنزله ملف مضغوط بداله.

الخطوات

1. نزّل المشروع:

```
git clone https://github.com/ELD11q/-ahs.git
cd -ahs/lunar-landing-navigator
```

بما إن الـ repo خاص، GitHub بيطلب منك تسجل دخول. وتقدر بدل كذا تضغط **Code ← Download ZIP** من صفحة الـ repo.

2. سوّ بيئة بايثون خاصة بالمشروع (مستحسن):

```
# ويندوز
python -m venv .venv
.venv\Scripts\activate

# ماك / لينكس
python3 -m venv .venv
source .venv/bin/activate
```

3. ثبّت المكتبات:

```
pip install -r requirements.txt
```

المكتبات هي: `numpy` و `scipy` للحسابات، و `matplotlib` و `pillow` للصور، و `pytest` للاختبارات.

4. تأكد إن كل شي شغال:

```
python -m pytest -q
# المفروض يطلع: 8 passed
```

7. أول تشغيل (البيانات التجريبية)

```
python -m lunar_nav --demo
```

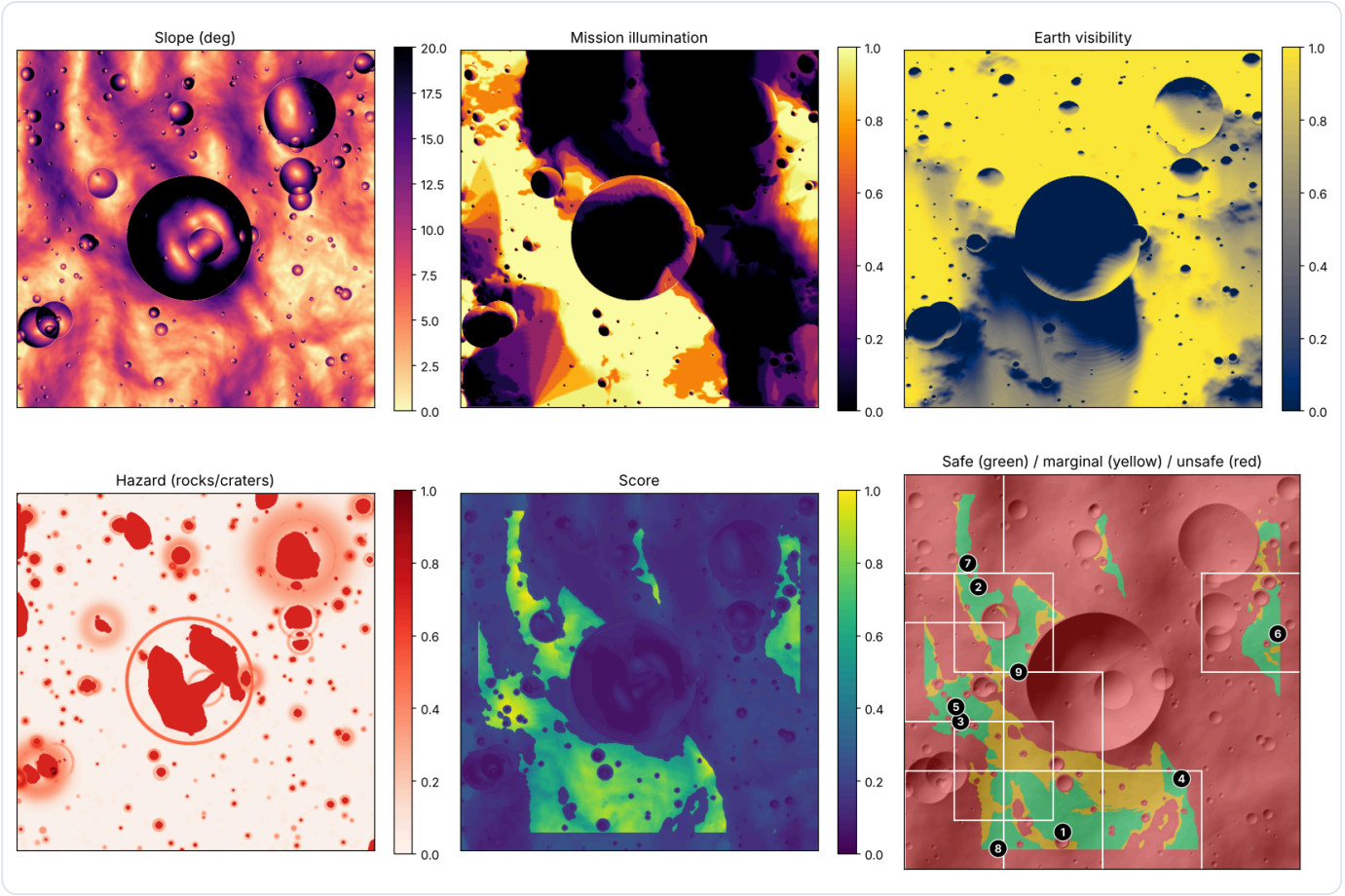
هذا يولّد منطقة قطبية تجريبية مساحتها 60×60 كم. فيها حفرة كبيرة عميقة جنب القطب (شبه حفرة Shackleton) أرضيتها مظلمة دايمًا، وسلسلة مرتفعات مضيئة، وحوالي 220 حفرة صغيرة، وحقول صخور. التحليل ياخذ تقريبًا 30 ثانية، ويطلع شي زي كذا:

```
[1/5] slope: 69.0% of area <= 10.0 deg
[2/5] illumination: mission mean 0.38, PSR 9.1% of area, Earth visible 0.67
[3/5] hazards: 151 craters detected
[4/5] classes: green 12.1%  yellow 7.7%  red 80.2%
[5/5] regions: 13 candidates -> top 9

Top regions:
 #  score   safe  slope  illum  earth  PSR km  lat/lon
 1   0.60    51%   1.5°    96%    59%    1.30  -89.175, 193.9E
 2   0.53    35%   0.3°   100%   100%    2.22  -89.248, 304.5E
 ...
```

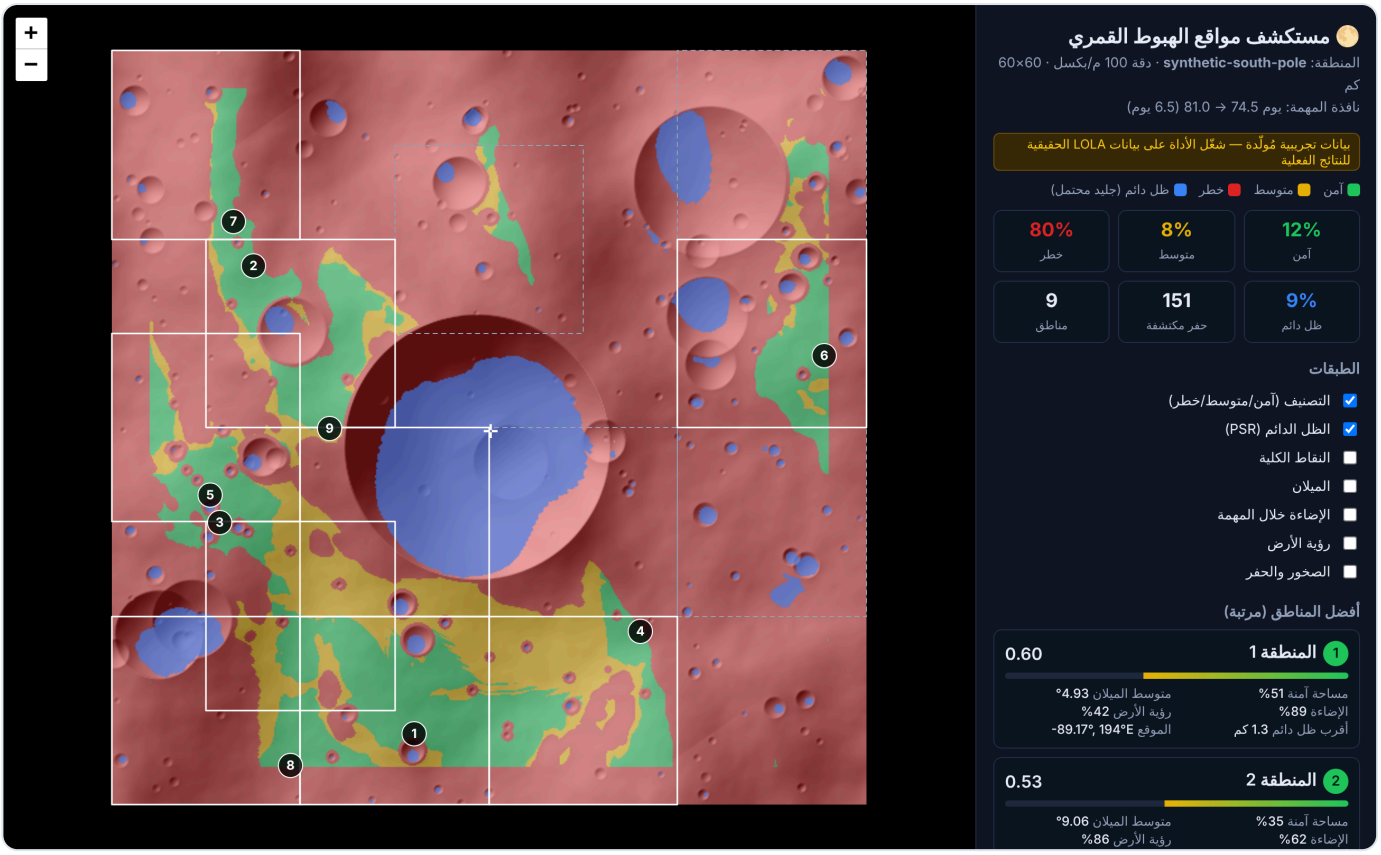
المخرجات (مجلد `output/`)

الملف	وش فيه
`landing_sites_map.html`	الخريطة التفاعلية، تفتحها بأي متصفح وتشتغل بدون إنترنت
`summary.png`	صورة فيها كل الطبقات جنب بعض
`ranked_regions.csv`	ترتيب المناطق، تقدر تفتحه بـ Excel
`ranked_regions.json`	نفس الترتيب بصيغة JSON، للبرامج
`layers.npz`	كل الطبقات كمصفوفات numpy، لو تبغى تحللها بنفسك



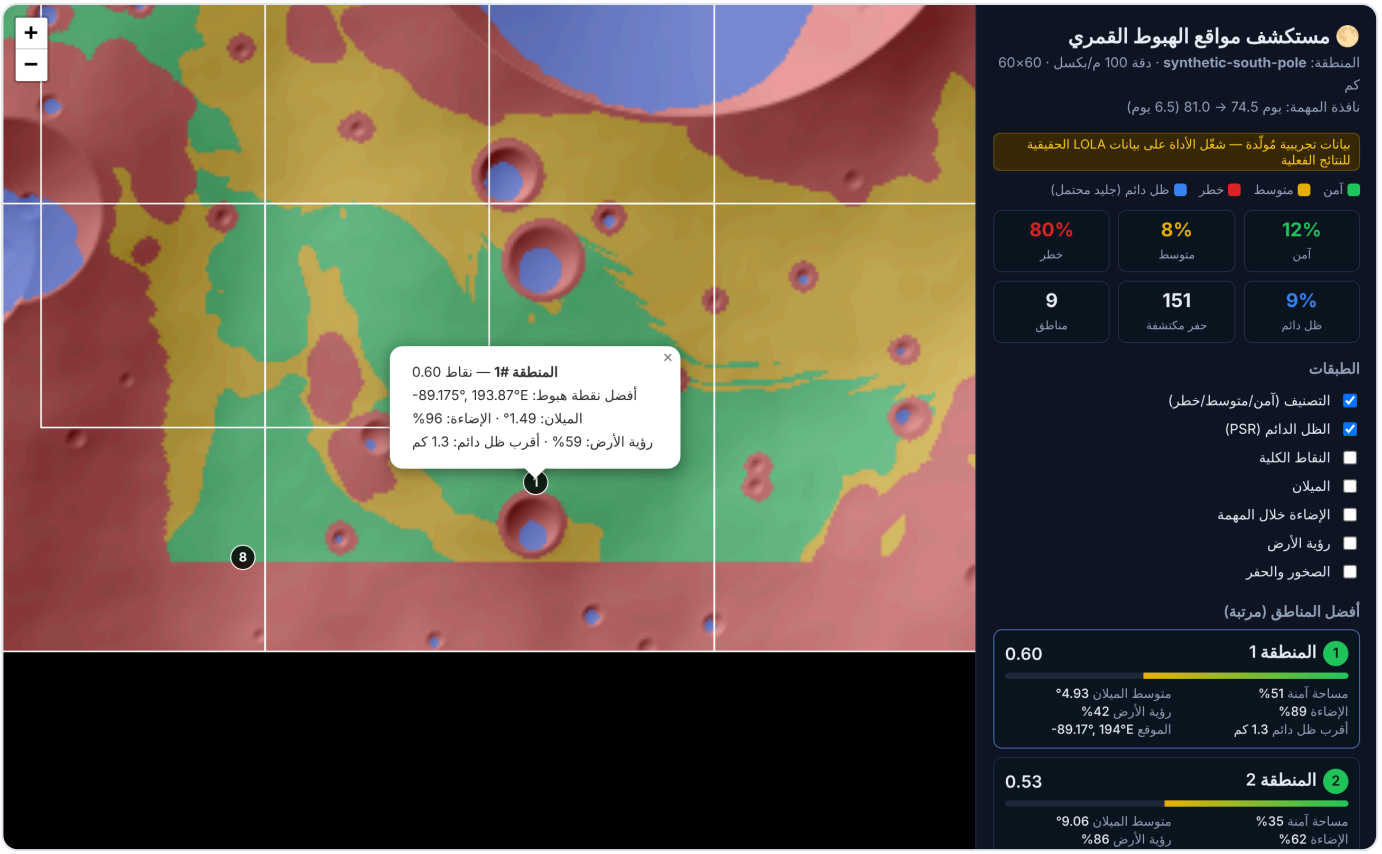
لوحة الملخص: الميلان، والإضاءة خلال المهمة، ورؤية الأرض، والخطر، والنقاط، والتصنيف النهائي مع أرقام المناطق

8. كيف تقرأ الخريطة والنتائج

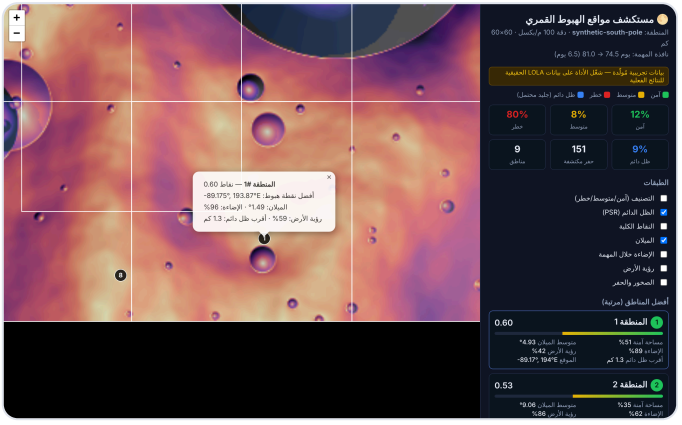


الشاشة الرئيسية: الخريطة على اليسار، واللوحة على اليمين فيها الإحصائيات والطبقات والمناطق المرتبة

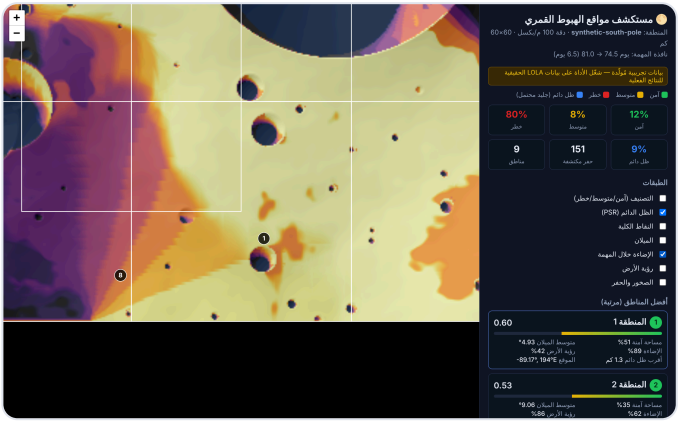
- **المربعات البيضاء:** أفضل 9 مناطق، وكل منطقة 15×15 كم.
- **المربعات المنقطة الرمادية:** المناطق المرشحة اللي انستبعدت (من 10 إلى 13).
- **الدوائر المرقمة:** أفضل نقطة هبوط داخل كل منطقة.
- **علامة ✛:** مكان القطب الجنوبي.
- اضغط على أي منطقة في القائمة، والخريطة تقرّب عليها وتطلع تفاصيلها.



بعد الضغط على المنطقة 1: الإحداثيات، والميلان، والإضاءة، ورؤية الأرض، والمسافة لأقرب ظل دائم



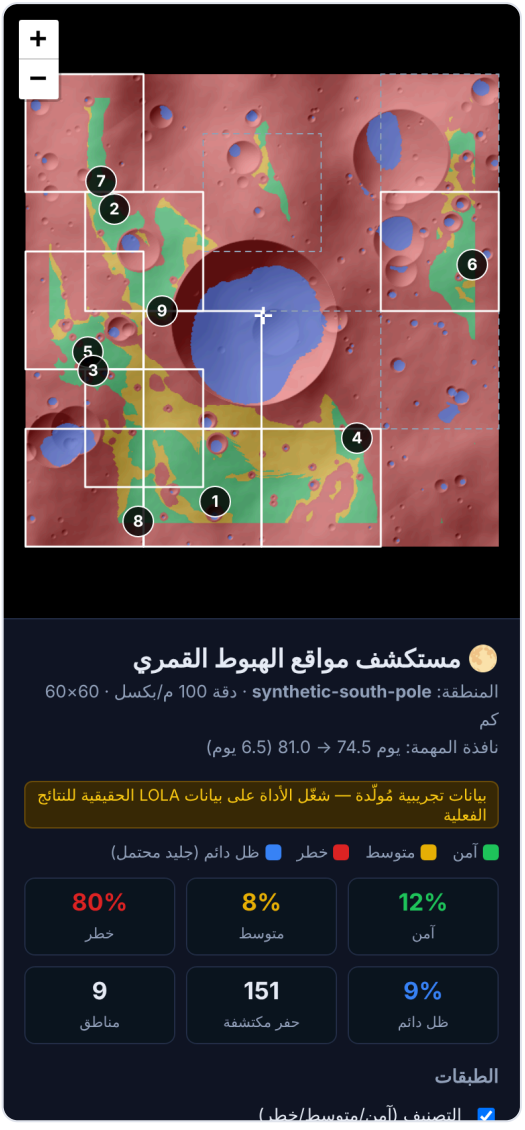
طبقة الميلان: الفاتح مستوي والغامق شديد الميلان



طبقة الإضاءة خلال المهمة: الأصفر مضيء دايمًا والأسود مظلم

معنى الأعمدة في الجدول

العمود	المعنى
score	نقاط المنطقة من 0 إلى 1. تُحسب من نسبة المساحة الآمنة (45%)، والمساحة المتوسطة (15%)، ومتوسط أفضل 5% من النقاط فيها (40%)
safe	نسبة المساحة الخضراء داخل المنطقة
slope / illum / earth	الميلان، ونسبة الإضاءة، ونسبة رؤية الأرض عند أفضل نقطة هبوط
PSR km	المسافة من نقطة الهبوط لأقرب ظل دائم
lat / lon	خط العرض والطول على القمر



الخريطة تشتغل على الجوال بعد

9. التشغيل على سيرفرات GitHub (GitHub Actions)

المشروع فيه ملف `.github/workflows/lunar-navigator.yml`. هذا الملف يخلي GitHub يشغّل النظام كامل على سيرفراته، بدون ما تحتاج جهازك.

متى يشتغل تلقائي؟

- كل ما ترفع تعديل على ملفات المشروع (push).
- كل ما تفتح Pull Request.

وش يسوي في كل تشغيل؟

1. يجهز سيرفر Ubuntu عليه Python 3.12 ويثبت المكتبات.
2. يشغّل الاختبارات الثمانية، وإذا فشل واحد يوقف ويعلمك.
3. يشغّل التحليل كامل.
4. يكتب جدول أفضل المناطق في صفحة نتيجة التشغيل نفسها.
5. يرفع الخريطة التفاعلية والصور والجداول كملف تنزله (Artifact) ويحتفظ فيه 30 يوم.

كيف تشغّله يدويًا وتشوف النتيجة

1. افتح الـ repo في GitHub واضغط تبويب **Actions**.
2. من القائمة اليسار اختر **Lunar Landing Navigator**.
3. اضغط **Run workflow**. تقدر تغيّر الإعدادات أو تخليها زي ما هي، وبعدين اضغط الزر الأخضر **Run workflow**.
4. انتظر دقيقتين تقريبًا لين تطلع علامة ✅.
5. افتح التشغيل، وبتلقى جدول النتائج في الصفحة، وتحته قسم **Artifacts** فيه ملف **landing-sites**. نزّله وفك الضغط، وافتح `landing_sites_map.html`.

الإعدادات اللي تقدر تغيّرها وقت التشغيل اليدوي

الإعداد	الوظيفة
`dem_url`	رابط ملف ارتفاعات حقيقي من ناسا. إذا خليته فاضي يشتغل على البيانات التجريبية
`pixel_size`	دقة البكسل بالمتر للملف الحقيقي
`downsample`	كم مرة يصغّر الملف عشان يكون الحساب أسرع
`dem_scale`	معامل يحوّل القيم في الملف إلى متر
`max_slope`	أقصى ميلان مسموح بالدرجات
`seed`	رقم يغيّر شكل البيانات التجريبية

ميزة كبيرة لسيرفرات GitHub إنها متصلة بالإنترنت بدون قيود، فتقدر تنزّل بيانات ناسا الحقيقية مباشرة. يكفي تحط رابط الملف في `dem_url` .

عن الـ repo الخاص: GitHub Actions تشتغل على الـ repo الخاص، والحساب المجاني فيه 2000 دقيقة تشغيل في الشهر، وكل تشغيل ياخذ تقريبًا 2 إلى 3 دقايق على البيانات التجريبية. أما نشر الخريطة كموقع عام عن طريق GitHub Pages فيحتاج repo عام أو حساب مدفوع، عشان كذا النتائج تنزل كملف.

10. التشغيل على بيانات ناسا الحقيقية

أ) وين تلقى البيانات

```
python scripts/download_nasa_data.py --list
```

البيانات	المصدر
خرائط ارتفاع LOLA للقطبين	pds-geosciences.wustl.edu/lro/lro-l-lola-3-rdr-v1/lrolol_1xxx/data/lola_gdr/polar/
ارتفاعات عالية الدقة وخرائط إضاءة للقطب الجنوبي	pgda.gsfc.nasa.gov/products/
خرائط الصخور من Diviner	pds-geosciences.wustl.edu/missions/lro/diviner.htm
صور LROC NAC (لـ YOLO)	wms.lroc.asu.edu/lroc/rdr_product_select

ب) نصايح لاختيار الملف

- ابدأ بملف دقته 20 إلى 80 متر للبكسل للقطب الجنوبي (85S أو 80S). ملفات الـ 5 متر حجمها عدة جيجابايت.
- ملفات LOLA بصيغة `.IMG` يجي معها ملف `.LBL`. اقرأ فيه `SCALING_FACTOR`، ومرّر القيمة اللي تحوّل الأرقام لمتر في `--dem-scale`.
- ملفات GeoTIFF أسهل. إذا ثبّت `rasterio` النظام يعرف دقة البكسل من الملف نفسه.

ج) التشغيل

```
python scripts/download_nasa_data.py <رابط-الملف> -o data/
pip install rasterio          # اختياري لكنه مستحسن مع GeoTIFF
python -m lunar_nav --dem data/<اسم-الملف>.tif --pixel-size 20 --downsample 4

# مع خريطة صخور وخريطة إضاءة جاهزة من ناسا:
python -m lunar_nav --dem data/dem.tif --pixel-size 20 --downsample 4 \
    --rocks data/rocks.tif --illum data/avg_illum.tif
```

> النظام يفترض إن الخريطة بإسقاط قطبي ستيريوغرافي، والقطب في منتصفها، وخط الطول 0 لفوق. هذا هو التنسيق المعتاد في ملفات LOLA القطبية. إذا القطب في مكان ثاني، حدده بـ `--pole ROW COL`.

11. كيف يشتغل النظام من الداخل

الخطوة 1: الميلان (`terrain.py`)

لكل بكسل نحسب فرق الارتفاع مع جيرانه في الاتجاهين، ونحوّله لزاوية:

```
slope = arctan( √( (∂z/∂x)² + (∂z/∂y)² ) )
```

ونحسب بعد **الخشونة**: نشيل الاتجاه العام للأرض ونقيس الانحراف المعياري للتضاريس الصغيرة. المنطقة الخشنة حتى لو ميلانها العام قليل فيها خطر.

الخطوة 2: الأفق (`illumination.py`)

هذي أهم خطوة. في القطب الجنوبي الشمس ما ترتفع أكثر من حوالي 1.5° فوق الأفق، فأي تل صغير يقدر يحجبها. عشان كذا لكل بكسل نرمي أشعة في **32 اتجاه** لين مسافة 40 كم، ونسجل أعلى زاوية أفق في كل اتجاه. وندخل في الحساب **انحناء القمر**، لأن الجبل البعيد ينزل تحت الأفق بمقدار:

```
drop = d² / (2R)        R = 1737.4 km  (نصف قطر القمر)
```

الخطوة 3: حركة الشمس والأرض

- **الشمس** تلف حول سماء القطب مرة كل 29.5 يوم، وارتفاعها يتغير بين −1.54° و +1.54° على مدار 346.6 يوم، وهذي "فصول" القمر.
- **الأرض** ثابتة تقريبًا باتجاه الوجه القريب، لكنها تتمايل (libration) ±6.7° فوق وتحت الأفق كل 27.2 يوم تقريبًا.
- كل ما بعدت النقطة عن القطب، تتغير زاوية الجسم في سماها بمقدار: colatitude × cos(Δazimuth).

بعدها نحرك الشمس والأرض كل 3 ساعات طول مدة المهمة، ونقارن ارتفاعهم بالأفق اللي حسبناه، ونطلع نسبة الوقت اللي يكونون فيها ظاهرين.

وعشان نختار **وقت المهمة**، النظام يدوّر على أفضل 6.5 يوم في السنة: يكون فيها صيف القطب الجنوبي، والأرض فوق الأفق أغلب الوقت. نفس الشي اللي تسويه ناسا لما تختار نافذة الإطلاق.

الخطوة 4: الظل الدائم (PSR)

نحسب الإضاءة على مدار 6 سنوات قمرية، وأي بكسل متوسط إضاءته أقل من **0.5%** ينحسب ظل دائم. هذي الأماكن حرارتها تقريبًا −230°C، وهي اللي يُحتمل فيها جليد الماء. بعدين نحسب لكل بكسل المسافة لأقرب ظل دائم.

الخطوة 5: الحفر والصخور (`hazards.py`)

بدون ذكاء اصطناعي: نطرح من الخريطة نسخة منعّمة منها على 3 مقاييس مختلفة، والأماكن اللي تطلع أوطى بوضوح من محيطها هي حفر. وبعدها نجمع الحفر مع الخشونة ومع خريطة صخور Diviner لو موجودة، ويطلع لنا مؤشر خطر من 0 إلى 1.

الخطوة 6: النقاط والتصنيف (`scoring.py`)

كل معيار يتحول لرقم من 0 (سيء) إلى 1 (ممتاز):

المعيار	التحويل	الوزن الافتراضي
الميلان	1 − الميلان ÷ 10°	30%
الإضاءة	نسبة الإضاءة خلال المهمة	25%
رؤية الأرض	نسبة الوقت اللي الأرض ظاهرة فيه	15%
القرب من الظل الدائم	1 إذا المسافة أقل من 2 كم، وينزل بالتدريج لين يوصل 0 عند 15 كم	15%
الصخور والحفر	1 − مؤشر الخطر	15%

النقاط الكلية = مجموع (الوزن × نقاط المعيار)

وبعدها التصنيف:

- **أحمر:** ميلان فوق 10°، أو ظل دائم، أو إضاءة المهمة أقل من 30%، أو خطر صخور فوق 0.6. ونوسّعه 300 م (قطع الهبوط)، ونستبعد بعد أطراف الخريطة (3 كم) لأن الأفق اللي وراها مو معروف.
- **أصفر:** ميلان بين 8° و 10°، أو رؤية الأرض أقل من 30%، أو النقاط الكلية أقل من 0.55.
- **أخضر:** كل اللي باقي.

الخطوة 7: المناطق (`regions.py`)

نمرر نافذة 15×15 كم على الخريطة بخطوة 7.5 كم، ونحسب نقاط كل نافذة. بعدين ناخذ أفضل 13 نافذة بشرط ما تتداخل أكثر من 25% وما تتكرر فيها نفس نقطة الهبوط، ومنها نحتفظ بأفضل 9. ونحوّل مكان أفضل نقطة لخط عرض وطول بمعادلة الإسقاط القطبي العكسي.

12. تعديل الشروط والأوزان

من سطر الأوامر

```
# ميلان أقصى 8 درجات بدل 10
python -m lunar_nav --demo --max-slope 8

# تعطي الإضاءة أهمية أكبر
python -m lunar_nav --demo --weights '{"illumination":0.4,"slope":0.3}'

# تحدد يوم بداية المهمة بنفسك
python -m lunar_nav --demo --mission-start 120

# بيانات تجريبية مختلفة
python -m lunar_nav --demo --seed 42 --size 800
```

من الكود (`config.py`)

كل الأرقام في مكان واحد. مثلًا `landing_ellipse_radius_m` لحجم قطع الهبوط، و `region_size_m` لحجم المنطقة، و `n_final_regions` لعدد المناطق النهائية، و `psr_ideal_distance_m` للمسافة المثالية للظل الدائم.

13. كشف الحفر والصخور بـ YOLO

خريطة الارتفاع دقتها عشرات الأمتار، فما تشوف الصخور الصغيرة. عشان كذا النظام فيه دالة جاهزة `detect_hazards_yolo` تستخدم نموذج YOLO على صور كاميرا LROC.

1. ثبّت المكتبة: `pip install ultralytics`
2. جهّز بيانات تدريب، يعني صور NAC عليها مربعات حول الحفر والصخور. تقدر تبدأ من قاعدة بيانات حفر Robbins أو تعلّم الصور بنفسك بأداة زي Roboflow أو CVAT.
3. درّب النموذج:

```
yolo detect train data=craters.yaml model=yolov8n.pt imgsz=640 epochs=100
```

4. شغّل النظام مع النموذج:

```
python -m lunar_nav --dem data/dem.tif --pixel-size 20 \
    --yolo-weights runs/detect/train/weights/best.pt \
    --yolo-image data/nac_mosaic.tif
```

> صورة الكاميرا لازم تكون مطابقة لنفس منطقة خريطة الارتفاع. النظام يغيّر حجمها تلقائي لين تطابق حجم الخريطة.

14. الاختبارات

```
python -m pytest -q
```

الاختبار	وش يتأكد منه
test_slope_of_known_plane	سطح مايل 12° بالضبط، والنظام يقيسه 12°
…_test_flat_terrain	الأرض المستوية ما فيها أفق، ومضيئة في صيف القطب
…_test_deep_pit_floor	قاع حفرة عميقة يطلع ظل دائم، وما حولها مضيء
…_test_latlon_of_pole	تحويل الإحداثيات صحيح، والقطب يطلع −90°
…_test_mission_window	نافذة المهمة المختارة فيها الشمس والأرض فوق الأفق
test_pipeline_end_to_end	النظام كامل يشتغل، والترتيب صحيح، وما فيه نقطة هبوط حمراء
test_steep_areas_are_red	كل منطقة ميلانها فوق 10° أو فيها ظل دائم تطلع حمراء
test_html_export	الخريطة تنحفظ وتشتغل بدون إنترنت

15. حدود النظام وأفكار للتطوير

الحدود الحالية

- هذا نظام تعليمي وبحثي، **مو أداة طيران معتمدة**.
- حساب الإضاءة مبسّط: 32 اتجاه للأفق، والشمس قرص بتقريب خطي، وزاوية الشمس تقريبية وتكون دقيقة للمناطق القريبة من القطب (حوالي 5°).
- البيانات التجريبية مولّدة عشوائيًا. الأرقام الحقيقية تطلع بس مع بيانات LOLA.
- أطراف أي خريطة ما تنحسب، لأن الجبال اللي وراها مو معروفة. عشان كذا حمّل منطقة أكبر شوي من اللي تحتاجه.

أفكار للتطوير

- استخدام خرائط الإضاءة الرسمية من ناسا (PGDA) مع الخيار `--illum` بدل الحساب المبسّط.
- زيادة عدد الاتجاهات لـ 180 أو أكثر، وتسريع الحساب بـ GPU.
- تدريب YOLO فعلي على صور NAC.
- حساب أطول فترة متواصلة في الظلام لكل نقطة، لأن البطاريات لازم تتحملها.
- تخطيط مسار للعربة الجوالة من نقطة الهبوط للظل الدائم.

16. حل المشاكل الشائعة

المشكلة	الحل
`python: command not found`	استخدم `python3`، وفي ويندوز أعد تثبيت بايثون وفعّل "Add to PATH"
`ModuleNotFoundError`	تأكد إنك مفعّل البيئة (`.venv`)، وشغّل `pip install -r requirements.txt`
`Pixel size unknown`	أضف `--pixel-size 20` بالرقم الصحيح للملف، أو ثبّت `rasterio`
التشغيل بطيء أو الذاكرة ما تكفي	كبّر `--downsample` ، أو قلّل `--azimuths` لـ 16
الارتفاعات غريبة (كبيرة جدًا أو صغيرة جدًا)	ارجع لملف `.LBL` وصحح قيمة `--dem-scale`
تشغيل GitHub Actions فشل ❌	افتح التشغيل واضغط على الخطوة الحمراء وشوف السجل. إذا كان الخطأ في التنزيل، تأكد إن رابط `dem_url` صحيح ويفتح مباشرة
كل الخريطة حمراء	غالبًا وقت المهمة في شتاء القطب، أو قيمة `dem-scale` غلط. جرّب `--mission-start` بقيمة ثانية

17. المصادر

المصادر اللي جات مع فكرة المشروع

1. NASA Scientific Visualization Studio: الإضاءة في القطب الجنوبي للقمر
https://svs.gsfc.nasa.gov/vis/a000000/a003500/a003587/script_9206_00.html
2. NASA Technical Reports Server: 20250011660
https://ntrs.nasa.gov/citations/20250011660
3. Aerospace America (AIAA): "The Choice"، عن اختيار مناطق هبوط Artemis III
https://aerospaceamerica.aiaa.org/features/the-choice/
4. arXiv 2406.19863
https://arxiv.org/pdf/2406.19863

مراجع علمية للطريقة

5. Mazarico, E., Neumann, G.A., Smith, D.E., Zuber, M.T., Torrence, M.H. (2011). *Illumination conditions of the lunar polar regions using LOLA topography.* Icarus 211, 1066–1081. هذا الأساس اللي بنينا عليه حساب الأفق والإضاءة.
6. Smith, D.E. et al. (2010). *The Lunar Orbiter Laser Altimeter Investigation on the Lunar Reconnaissance Orbiter Mission.* Space Science Reviews 150.
7. Bandfield, J.L. et al. (2011). *Lunar surface rock abundance and regolith fines temperatures derived from LRO Diviner Radiometer data.* JGR Planets 116. عن طريقة خرائط الصخور من Diviner.
8. Robinson, M.S. et al. (2010). *Lunar Reconnaissance Orbiter Camera (LROC) Instrument Overview.* Space Science Reviews 150.
9. Robbins, S.J. (2019). *A New Global Database of Lunar Impact Craters >1–2 km.* JGR Planets 124. doi:10.1029/2018JE005592
10. NASA (2024). *NASA Provides Update on Artemis III Moon Landing Regions*. nasa.gov.

مصادر البيانات

- PDS Geosciences Node: LOLA GDR Polar: pds-geosciences.wustl.edu
- NASA GSFC Planetary Geodesy Data Archive (PGDA): pgda.gsfc.nasa.gov
- LROC Data Products: wms.lroc.asu.edu

الأدوات المستخدمة

Python و NumPy و SciPy و Matplotlib و Pillow و Leaflet (الخرائط، رخصة BSD-2) و Ultralytics YOLO (اختياري) و GitHub Actions. والخط المستخدم Cairo (رخصة OFL).

> ملاحظة أمانة: في البيئة اللي انبنى فيها المشروع ما كان فيه وصول لمواقع ناسا، فالنتائج والصور في هذا الدليل كلها من البيانات التجريبية. والروابط اللي فوق هي البوابات الرسمية، لكن مسارات الملفات داخلها ممكن تتغير مع الوقت.